

Robotic AI Project - Monthly Progress Summary

Project: Piper 6-DOF Robotic Arm Vision-Guided Manipulation System **Date Range:** January 2026 (Approximately 4 weeks) **Author:** Robotics UROP Team

Executive Overview

This month-long project focused on developing a complete vision-guided robotic manipulation system using the Piper 6-DOF robotic arm, RealSense depth camera, and RGI rotating gripper. The work spanned multiple areas including ROS2 integration, imitation learning with LeRobot, camera calibration, and custom control interfaces. The project aimed to replicate capabilities similar to the ArmPi Ultra system while adding advanced features like trajectory recording, web-based control, and machine learning-based policy deployment.

The major accomplishments include: establishing reliable CAN bus communication with the Piper arm, creating comprehensive gamepad control systems, implementing hand-eye camera calibration, setting up the LeRobot framework for imitation learning, building a depth camera GUI viewer, and developing a complete trajectory recording/playback system with web-based remote control.

Week 1: Environment Setup & ROS2 Integration

Objectives

- Set up development environment
- Establish communication with Piper arm
- Configure ROS2 workspace

Accomplishments

ROS2 Jazzy Installation & Configuration

- Installed ROS2 Jazzy distribution
- Configured workspace at `~/ros2_piper_ws`
- Set up environment variables and sourcing

Piper SDK Integration

- Installed `piper_sdk` for CAN bus communication
- Configured CAN interface (`can0`) at 1Mbps bitrate
- Tested basic arm movements and joint control

Documentation Created

File	Description
<code>ROS_SETUP_GUIDE.md</code>	ROS2 installation instructions
<code>HOW_TO_RUN.md</code>	Quick start guide
<code>QUICK_SETUP.md</code>	Environment setup steps
<code>BUILD_AND_RUN_ROS2.md</code>	Building ROS2 packages

Scripts Created

- `install_ros_and_setup.sh` - Automated ROS2 installation
- `setup_ros_workspace.sh` - Workspace configuration
- `complete_ros2_setup.sh` - Full setup script

Technical Details

```
# CAN bus setup
sudo ip link set can0 up type can bitrate 1000000

# Piper connection
piper = C_PiperInterface_V2("can0")
piper.ConnectPort()
piper.EnablePiper()
```

Week 2: URDF, RViz2 & Kinematics

Objectives

- Load Piper URDF into RViz2
- Implement inverse kinematics
- Create sim-to-real bridge

Accomplishments

URDF Visualization

- Located URDF files in `piper_description` package
- Created launch files for RViz2 visualization
- Fixed joint limits and collision meshes

Piper Kinematics Package

Created `piper_kinematics` ROS2 package with: - `piper_ik_node.py` - Inverse kinematics solver - `interactive_pose_marker.py` - Interactive pose control - Launch files for various configurations

Sim-to-Real Bridge

- `sim_to_real_bridge.py` - Bridges RViz commands to real robot
- Subscribes to `/target_pose` topic
- Publishes joint commands to physical arm

Documentation

File	Description
<code>HOW_TO_USE_RVIZ2.md</code>	RViz2 usage guide
<code>LOAD_ROBOT_IN_RVIZ2.md</code>	Loading URDF in RViz
<code>RVIZ2_SETUP_GUIDE.md</code>	RViz configuration
<code>WHERE_URDF_IS_USED.md</code>	URDF file locations
<code>ROS2_WORKSPACE_EXPLAINED.md</code>	Workspace structure

Week 3: Camera Integration & Calibration

Objectives

- Integrate RealSense depth camera
- Perform hand-eye calibration
- Create depth camera GUI

Accomplishments

RealSense Camera Setup

- Installed `realsense2_camera` ROS2 package
- Configured RGB and depth streams
- Created camera launch scripts

Camera Troubleshooting

- Fixed IR emitter errors
- Resolved USB bandwidth issues
- Created reset scripts for camera recovery

Script	Purpose
<code>fix_realsense_ir_error.sh</code>	Fix IR stream errors
<code>force_reset_camera.sh</code>	Force camera reset
<code>reset_realsense_camera.py</code>	Python reset utility
<code>launch_realsense_fixed.sh</code>	Launch with fixes

Hand-Eye Calibration

Created comprehensive calibration system in `piper/calibration/` :

Files Created: - `hand_eye_calibration.py` - Main calibration script -
`generate_charuco_board.py` - ChArUco board generator - `generate_charuco_A3.py` - A3
size board - `verify_calibration.py` - Calibration verification -
`coordinate_transform_node.py` - Coordinate transforms

Calibration Outputs: - `camera_intrinsics.yaml` - Camera parameters -
`hand_eye_calibration.yaml` - Transform matrix - `charuco_board_A4.png` /
`charuco_board_A3.png` - Printed targets

Depth Camera GUI

Created `depth_camera_gui.py` - Tkinter-based viewer with: - Real-time RGB and depth
visualization - Depth colormap selection (JET, HOT, TURBO, etc.) - Depth range
sliders - Statistics display (min/max/mean depth)

Week 4: Gamepad Control & Object Detection

Objectives

- Create gamepad control system
- Implement cube detection
- Build pick-and-place system

Accomplishments

Gamepad Control ([piper/gamepad/](#))

- `gamepadecontrol.py` - Main gamepad controller
- `main.py` - Standalone gamepad control
- `test_controller.py` - Controller testing
- `gripper_test.ipynb` - Jupyter notebook for gripper testing

Control Mapping: | Control | Function | |-----|-----| | Left Stick | Joint 1-2 | | Right Stick | Joint 3-4 | | D-Pad | Joint 5-6 | | LB/RB | Speed control | | Back/Start | Gripper open/close | | R3 | Home position |

Cube Detection ([piper/cubeAndLineDet/](#))

- Enhanced `cube_det.cpp` for cube detection
- Line detection in `line_det.cpp`
- ROS2 integration for object detection
- Created check and run scripts

Pick and Place

- `pick_and_place.py` - Basic pick and place
- `object_detection_pick_place.py` - Vision-guided manipulation
- `pick_and_place_calibrated.py` - Using calibration data

Week 4-5: LeRobot Imitation Learning

Objectives

- Set up LeRobot framework
- Collect demonstration data
- Train and deploy ACT policy

Accomplishments

LeRobot Framework Setup

Created `lerobot_piper_real/` directory with complete framework:

Core Components: - Modified LeRobot for Piper arm compatibility - Custom robot configuration files - Camera integration (RealSense support)

Data Collection System

- `record_dataset.py` - Demonstration recording
- Gamepad-based teleoperation
- Parquet format with embedded images
- 30 FPS recording at 2 minutes per episode

Training Pipeline

- `train_act.py` - ACT policy training
- Configurable hyperparameters
- Checkpoint saving system
- TensorBoard logging

Deployment

- `deploy_policy.py` - Policy deployment
- Real-time inference on robot
- Camera observation input

Documentation

- `SOP_TRAINING_GUIDE.md` - Complete training guide
- Step-by-step instructions
- Troubleshooting guide

Week 5: RGI Gripper Integration

Objectives

- Integrate RGI rotating gripper
- Create trajectory recording system
- Build web control interface

Accomplishments

RGI Gripper Control (`rgi_gripper/`)

Created complete gripper integration:

Files: - `play_and_adjust.py` - Main control script - `rotation_arm_gamepad.py` - Combined arm+gripper control - `web_controller.py` - Web interface - `gamepad_control.py` - Standalone control - `rgi_connection.py` - Low-level communication

Features: - Modbus RTU protocol over serial - Thread-safe communication - Open/close/rotate commands - Position feedback

Trajectory System

- Recording with teach mode
- JSON storage for multiple named trajectories
- Full management: record, play, rename, delete, view
- Position actions menu (rotate, open, close)

Web Interface

Flask-based web controller with: - Live Orbbec camera feed - Real-time joint display - Remote control buttons - Trajectory playback - WebSocket updates

CAN Bus Troubleshooting

Issues Resolved

Throughout the project, multiple CAN bus issues were encountered and resolved:

Documentation Created

File	Description
<code>CAN_TROUBLESHOOTING.md</code>	Comprehensive troubleshooting
<code>CAN_DEBUG_FIX.md</code>	Debug procedures

Scripts Created

- `diagnose_can_connection.py` - Connection diagnostics

- `debug_can_send.py` - Send debugging
- `fix_enable.py` - Enable fix script
- `test_can_rate.py` - Rate testing

Common Solutions

1. Power cycle arm for mode switch issues
2. Verify bitrate at 1000000
3. Check USB-CAN adapter connection
4. Reset CAN interface:

```
sudo ip link set can0 down && sudo ip link set can0 up type can bitrate 1000000
```

Project Structure Summary

```
Robotic-AI/
├── piper/
│   ├── calibration/           # Hand-eye calibration
│   ├── cubeAndLineDet/       # Object detection
│   ├── gamepad/              # Gamepad control
│   ├── manipulation/         # Pick and place
│   ├── piper_kinematics/     # ROS2 kinematics
│   ├── recordAndPlayPos/     # Position recording
│   └── recordAndPlayTraj/     # Trajectory recording
├── rgi_gripper/              # RGI gripper integration
│   ├── play_and_adjust.py    # Main control
│   ├── web_controller.py     # Web interface
│   └── saved_positions.json  # Trajectories
├── lerobot_piper_real/       # Imitation learning
│   ├── record_dataset.py
│   ├── train_act.py
│   └── deploy_policy.py
├── piper_sdk/                # Piper SDK
└── [Documentation Files]     # Various .md files
```

Key Metrics

Metric	Value
Python Files Created	~50+

Metric	Value
Documentation Files	~25+
Shell Scripts	~15+
ROS2 Packages	3
Total Lines of Code	~10,000+

Technologies Used

Category	Technologies
Robot Control	Piper SDK, CAN bus
Vision	RealSense, OpenCV, Orbbec
ROS2	Jazzy, RViz2, tf2
Machine Learning	LeRobot, ACT, PyTorch
Web	Flask, Flask-SocketIO, WebSockets
Communication	Modbus RTU, Serial
UI	Tkinter, Pygame

Future Work

1. **Object Detection Improvement** - Better accuracy for multiple objects
2. **Voice Commands** - Natural language control
3. **Path Planning** - Collision-free trajectories
4. **Multi-arm Coordination** - Dual-arm setup
5. **Mobile Base Integration** - LIMO robot integration
6. **Isaac Sim** - Full simulation environment

Conclusion

This month of work established a solid foundation for vision-guided robotic manipulation. The system now supports multiple control modalities (gamepad, web, teach mode), has comprehensive trajectory management, and includes the infrastructure for machine learning-based policy deployment. The modular architecture allows for easy extension and the documentation ensures reproducibility.

Generated: January 30, 2026